

Gift / Points Sharing — API Procedures Guide (for C# API)

This guide explains the stored procedures you will call from your C# API, their purposes, inputs/outputs, and recommended endpoints with example payloads and error handling.

1) Procedures Overview

Procedure / Function	Purpose	When to Call
sp_send_share_request(owner_id, requester_id, requested_percent)	Create or update a PENDING share request	Recipient submits and changes their desire
sp_respond_share_request(request_id, owner_id, accept)	Owner accepts/rejects request. On accept, updates allocation and express to respo	Owner allocation and express to respo
sp_share_owner_amount(owner_id, amount_rs, note)	Transactional distribution: deducts from owner's wallet and credits recipients' logs	Owner (its adj) logs by activating logs
fn_total_percent_for_owner(owner_id) [FUNCTION]	Utility helper that returns total active % allocated by the display before accepting	Utility helper that returns total active % allocated by the display before accepting

Important Fix in Your Procedure

In your current code, the distribution procedure uses a table named `wallets`, but your schema defines `user\_wallets`. Replace all references to `wallets` with `user\_wallets` inside `sp\_share\_owner\_amount`.

```
-- Ensure single wallet per user and safe updates
ALTER TABLE user_wallets ADD CONSTRAINT ux_wallet_user UNIQUE (user_id);
```

2) Detailed Procedure Purpose & Behavior

2.1 sp_send_share_request(owner_id, requester_id, requested_percent)

Purpose: Insert a new pending request or update the % of an existing pending request from a recipient to an owner.
Key Validations: • owner_id != requester_id • 0 ≤ requested_percent ≤ 100 • Owner and requester must exist in users
Outcome: • Upsert into share_requests with status='pending'. Recommended API: POST /share-requests

```
CALL sp_send_share_request(20, 23, 20.00);
```

2.2 sp_respond_share_request(request_id, owner_id, accept)

Purpose: Owner accepts or rejects a specific pending request. On Accept: • Computes new total allocation for the owner (replaces existing recipient % if present). • Enforces total ≤ 100%; aborts if exceeded. • Upserts into share_allocations (links source_request_id). • Marks share_requests.status='accepted'. On Reject: • Marks share_requests.status='rejected'. Recommended API: POST /share-requests/{id}/respond

```
CALL sp_respond_share_request(3, 20, TRUE); -- accept
CALL sp_respond_share_request(4, 20, FALSE); -- reject
```

2.3 sp_share_owner_amount(owner_id, amount_rs, note)

Purpose: Execute the actual distribution transaction from owner to recipients per active allocations. Behavior: • Validates amount > 0. • SELECT ... FOR UPDATE on owner wallet row; checks sufficient balance. • Deducts owner's balance, credits each recipient (rounded to 2 decimals). • Inserts one row per recipient into transfers with percent snapshot. • Refunds any rounding residue back to owner for exact accounting. Atomicity: • Wrapped in a single transaction; uses cursor to iterate allocations. Recommended API: POST /wallets/{ownerId}/share

```
CALL sp_share_owner_amount(20, 200.00, 'Monthly share');
```

2.4 fn_total_percent_for_owner(owner_id)

Purpose: Read-only helper to sum active allocation percentages for an owner. Usage: • Call from UI or middleware to display 'current allocations total'. Recommended API: GET /owners/{ownerId}/allocations/total

```
SELECT fn_total_percent_for_owner(20) AS total_alloc;
```

3) Suggested REST Endpoints (C# implementation guide)

POST /share-requests

Body: { "owner_id": 20, "requester_id": 23, "requested_percent": 20.0 }

DB: CALL sp_send_share_request(?, ?, ?)

POST /share-requests/{id}/respond

Body: { "owner_id": 20, "accept": true }

DB: CALL sp_respond_share_request(?, ?, ?)

POST /wallets/{ownerId}/share

Body: { "amount_rs": 200.00, "note": "Monthly share" }

DB: CALL sp_share_owner_amount(?, ?, ?)

GET /owners/{ownerId}/allocations/total

DB: SELECT fn_total_percent_for_owner(?)

GET /owners/{ownerId}/allocations

DB: SELECT owner_id, recipient_id, percent_share FROM share_allocations WHERE owner_id=? AND is_active=1

GET /owners/{ownerId}/transfers

DB: SELECT * FROM transfers WHERE owner_id=? ORDER BY created_at DESC

4) Error Handling & HTTP Mapping

-- Procedures raise SIGNAL SQLSTATE '45000' with a message.

Map to HTTP:

400 Bad Request -> invalid input (percent/amount range, same owner/requester)

404 Not Found -> owner/requester/request not found

409 Conflict -> would exceed 100% allocation, insufficient owner balance

200/201 OK -> success

Sample JSON response on error:

```
{ "ok": false, "error": "Accepting this request would exceed 100% total allocation" }
```

5) Typical Flows (ASCII)

A) Request → Accept → Share

Recipient -> API: POST /share-requests

API -> DB: CALL sp_send_share_request

Owner -> API: POST /share-requests/{id}/respond (accept)

API -> DB: CALL sp_respond_share_request

Owner/System -> API: POST /wallets/{ownerId}/share

API -> DB: CALL sp_share_owner_amount

DB: updates user_wallets + transfers

6) C# Integration Notes

- Use parameterized calls: CALL sp_send_share_request(@owner_id, @requester_id, @requested_percent);
- Wrap each DB call in try/catch; return friendly error from e.sqlMessage.
- For distribution, you don't need your own transaction; the procedure handles it.
- Ensure user_wallets.user_id is UNIQUE to satisfy safe-update mode and 1:1 wallet.
- Consider a connection string setting 'Allow User Variables=true' if your driver requires it.

Appendix: Replace 'wallets' with 'user_wallets' in sp_share_owner_amount

-- Lock owner's wallet

```
SELECT balance_rs INTO v_balance FROM user_wallets WHERE user_id = p_owner_id FOR UPDATE;
```

-- Deduct from owner

```
UPDATE user_wallets SET balance_rs = balance_rs - p_amount_rs WHERE user_id = p_owner_id;

-- Credit recipients
INSERT INTO user_wallets (user_id, balance_rs)
VALUES (cur_recipient_id, cur_amt)
ON DUPLICATE KEY UPDATE balance_rs = balance_rs + VALUES(balance_rs);

-- Rounding residue back to owner
UPDATE user_wallets
SET balance_rs = balance_rs + (p_amount_rs - v_distributed)
WHERE user_id = p_owner_id;
```